Section 2: Association

Prediction: I think both products can still access the supplier information after del supplier because they have their own references to it.

Actual Result: Both products could still display the supplier information.

Explanation: Deleting the variable does not delete the Supplier object because the products still reference it.

Section 3: Aggregation

Prediction: I think the product will still work after being removed from Inventory A because it exists separately from the inventory.

Actual Result: The product could still be used and was successfully added to Inventory B.

Explanation: Removing a product from an inventory does not destroy the product itself.

Section 4: Composition

Prediction: I think there will be two OrderLine objects but only one Product object because both orders use the same product.

Actual Result: The two OrderLines were different objects, but they referenced the same Product.

Explanation: Each order creates its own OrderLine, but the Product can be shared between orders.

Section 6: Liskov Substitution Principle (LSP)

Question: Does the DigitalProduct code that raises NotImplementedError violate LSP?

Yes, because DigitalProduct should be able to perform the same basic operations as Product. I would keep the original update_stock() method so both product types can update their stock normally.

One expectation from Product:

Both PhysicalProduct and DigitalProduct should allow stock to be updated using update_stock().

Section 7: Polymorphism

Prediction: I think PhysicalProduct will display a shipping message, while DigitalProduct will display a digital delivery message.

Actual Result: Both products displayed their correct fulfilment messages.

Explanation: Both classes use the same method name, but each has its own implementation.

How does this demonstrate LSP and polymorphism?

Both product types can be passed into the same function without changing the code. Each product displays its own information correctly.

Section 8: UML Design and AI Review

One AI suggestion I accepted: I used aggregation between Inventory and Product because products can exist independently.

One suggestion I changed: I corrected the Order–OrderLine connection so the filled diamond is at Order, since Order owns its OrderLines.

Explanation: Association means objects are connected, aggregation means parts can exist independently, and composition means the whole owns its parts.